Available Online at www.ijcsmc.com/journal

International Journal of Computer Science and Mobile Computing

A Monthly Journal of Computer Science and Information Technology

ISSN 2320–088X
IMPACT FACTOR: 7.056

IJCSMC, Vol. 11, Issue. 8, August 2022, pg.1 – 8

Comparative Analysis of Various Software Development Life Cycle

[1]Gurudev Sawarkar; [2]Dr. Dipesh Rajput

[1]PhD Scholar, [2]Associate Professor

[1,2]Department of Computer Science and Engineering

[1,2]Swami Vivekananda University Sagar MP

DOI: https://doi.org/10.47760/ijcsmc.2022.v11i08.001

Abstract:

Software development life cycle is the process used by the software industry to design, build, and test high-quality software. SDLC's primary objective is to provide high-quality software that exceeds customer expectations. SDLC may also be referred to as Application Development Life Cycle. SDLC is not a technique; rather, it is a description of several phases.

Participate in software development from project design to deployment. Sustainment. These SDLC phases serve as a roadmap for project activities. In our in this study, we describe many SDLC models (Waterfall, Spiral, V-Model, Iterative,). Rapid Application Development Model).

Keywords: Waterfall, Spiral, V-Model, Iterative

1. INTRODUCTION

Today, software project planning management is an absolute necessity. A plan is comparable to a map. Software engineering is the process of creating high-quality software for computer systems. Software engineering is all about the quality-focused, process-driven, method-based, and tool-based software development methodologies and processes. The software development procedure specifies a structure comprising many activities and tasks. We can say the process specifies which software development activities and tasks are to be carried out (or technique of software development). The software planning management offers a technical starting point for implementing software processes, including Communication, Requirements Analysis, Analysis and Design modelling, Program development, Testing, and support. Software engineering life cycle SDLC is a methodical technique to completing the software development process on schedule while maintaining software quality. The system development life cycle is a collection of tasks that must be performed throughout system development; it is also known as the software development life cycle. Software development is segmented into a series of operations that enable any software development firm to efficiently manage the software product. Models of the software development life cycle employ a sequential method to finish the software development process. If the process is effective, the final output will likewise be effective, and the project will be successful.

The following considerations should be kept in mind by developers who are directly or indirectly involved in creating a quality software product.

- A quality focus.
- Process
- Methods
- Tools.

The software process model is the representation of process it presents the description of a process as.

- Specification.
- Design.
- Validation
- Evolution.

The software development life cycle is all about.

- Understanding the problem. i.e. (problem domain).
- Decide a plan for solution. i.e. (solution domain).
- Coding the planed solution.
- Test the actual program.
- Maintain the product

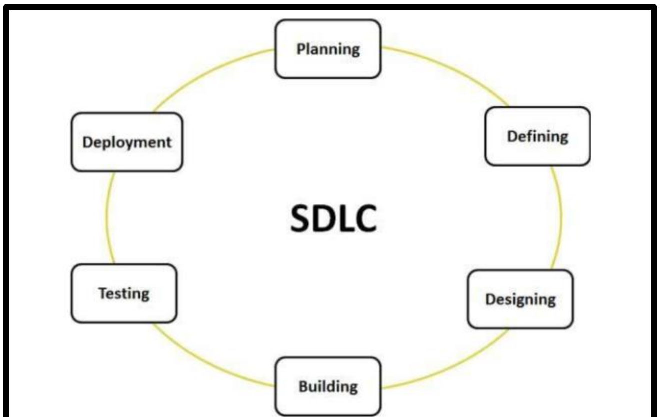

Figure 1 SDLC

THE SOFTWARE DEVELOPMENT LIFE CYCLE MODELS.

i. The Waterfall Model:

Royce came up with this concept around 1970. This is a typical software engineering process. Models like these have been around for a very long time and are still utilized in many government projects and significant corporations. Alternatively known as the linear sequential model or the traditional life cycle, this model is a linear progression. In this paradigm, the first step is Requirement Analysis, followed by Design, Implementation and Testing and Maintenance [1]. In order to prevent the same phase from repeating again, the phases are locked in place. The traditional form, the water fall, serves as the foundation for all subsequent models. According to the diagram below, the waterfall model is comprised of multiple distinct stages that do not cross over one another.

There is no going back on this one-way roadway. Once step "X" is complete, you can only go forward to phase "Y". There is no way to go backwards in this game.

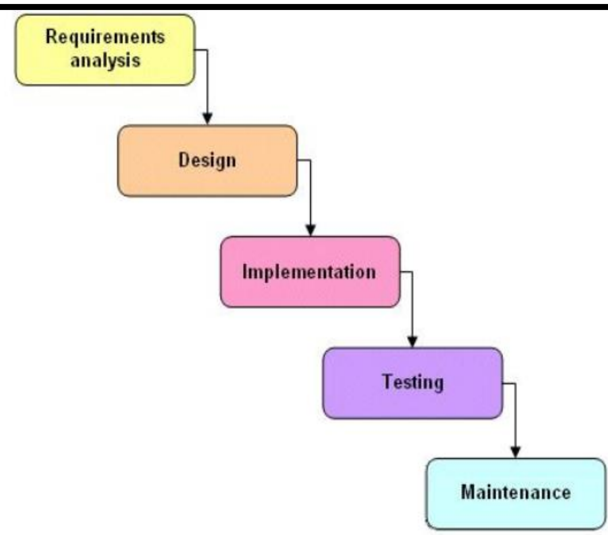

Figure 2 Waterfall model.

ii. Spiral Model:

Software projects use the rad model, a risk-driven process model generator, to integrate incremental approach with the methodical and regulated elements of the waterfall approach. [7, 8, and 9]

Advantages

Software is developed and managed in a strategic manner, and project tracking is simple and effective.

In other words, software may be generated earlier and users can see it sooner.

In addition, changes can be applied earlier in the life cycle and can even be implemented later on.

Finally, stringent approval and documentation control.

Spiral models may be used to produce a wide range of bespoke products.

Disadvantages

1. It's too expensive for smaller tasks, thus it's not a good choice for such.
2. The amount of paperwork necessary makes the development process extremely time consuming and difficult. 2
3. Due to the amount of paperwork that is necessary in the intermediate stages, the administration of this project is difficult.

4. A high level of skill is required for risk analysis.
5. The possibility of failing to fulfil the intended timetable.

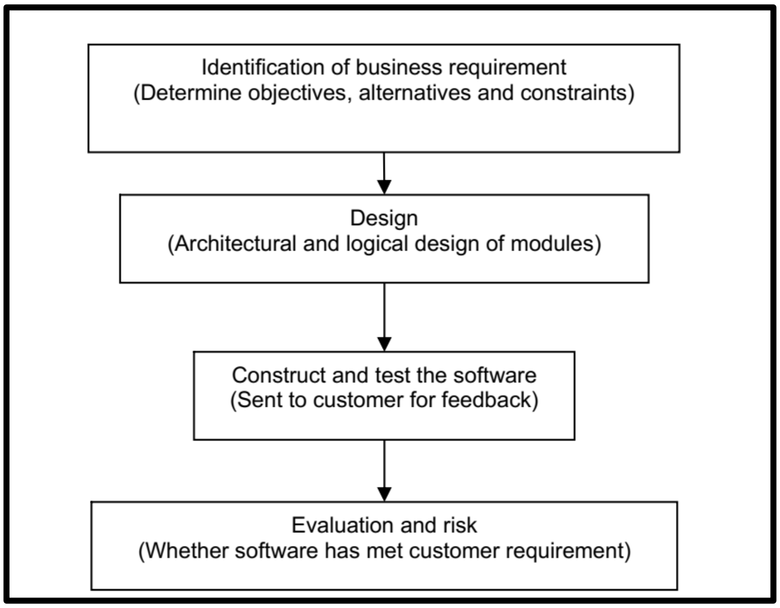

FIGURE 3: Represent the Steps Involved in Spiral Model.

iii. The Iterative Waterfall Model

A new model was needed because of the issues with the traditional waterfall paradigm. If you have an issue with waterfall, you may use an iterative method to solve it instead of using a more traditional method, such as using a traditional waterfall method. Projects under an iterative approach are broken into tiny segments, allowing the development team to move fast toward their objective and gain vital customer input. A micro waterfall technique is being used to break the project down into manageable chunks.

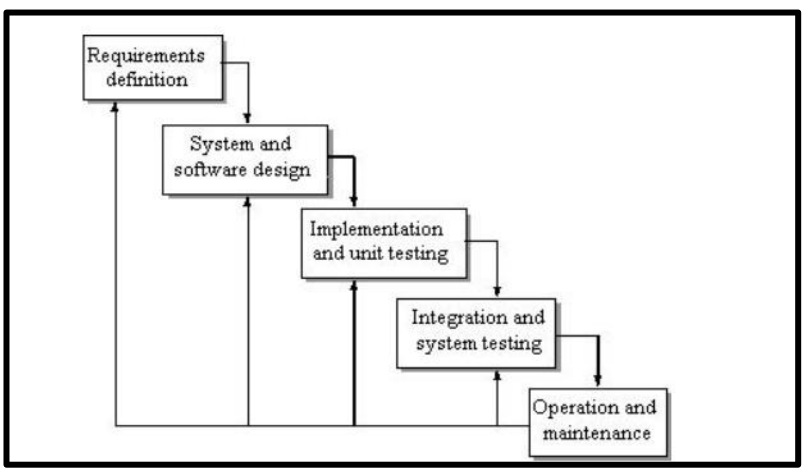

Figure 4 Iterative Waterfall model

Advantages:

I. Much better model of software process.

II. Client can get Feedback.

III. Used in that type of projects were requirements are not clear.

IV. Document driven process.

V. Works well on week teams.

Disadvantages:

I. Not easy to manage.

II. Not clear mile stones.

III. No stage is finished really.

iv. The V-Shaped Model:

An expansion of the waterfall model, this one. The linearity of the waterfall model is contrasted with the V-shaped model's bending of the coding stages upwards to create a classic V-shape [7].

All the phases of the development process and their accompanying testing phases. The V shaped approach emphasizes testing since testing is an important aspect of software development. V-shaped models are quite similar to waterfall models in that they both follow one after the other. It is extremely costly to go back to requirements and make adjustments to already-started projects, therefore they must be crystal defined before any work begins.

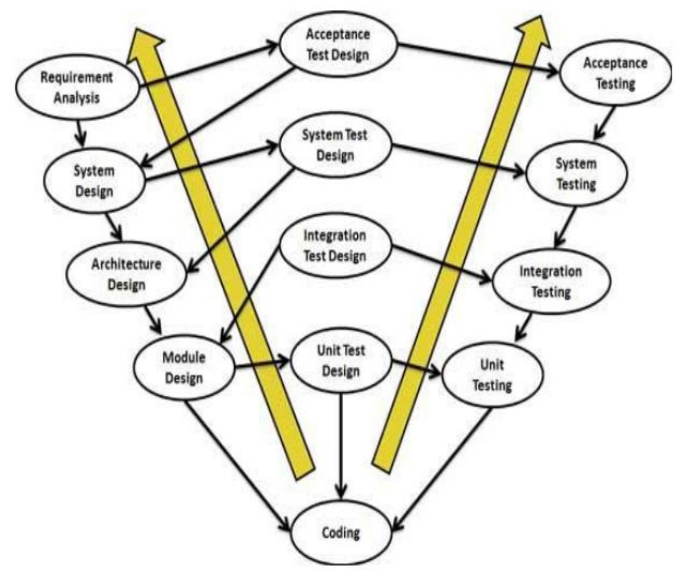

Figure 5 The v- shaped model.

Advantages:

I. High amount of risk analysis.

II. Good for critical projects.

III. Early production.

IV. Easy to manage due to rigidity of model.

V. Easy to understand.

Disadvantages:

I. Not good model for object oriented projects.

II. Not good for long and ongoing projects.

III. Not suitable were requirements have high risk of changing.

IV. Can be costly model.

V. Don't work well for small projects.

v. Comparative Analysis Of Five Models

MODEL/MERITS	WATERFALL MODEL	ITRATIVE WATERFALL MODEL	PROTOTYPE	V- SHAPED MODEL	SPIRAL MODEL
Success rate	Low	High	Good	High	High
Cost	Low	Low	High	Very high	High
Flexibility	Rigged	Less flexibility	Highly flexible	Little flexible	Flexible
Risk analysis	High	No risk analysis	Low	Low	Low
Cost control	Yes	No	No	Yes	Yes
Reusability	Limited	Yes	Weak	Low	Yes
Risk analysis	Only at beginning	Low	No risk analysis	Low	Yes
Implementation time	Long	Less	Less	Less	Depends on project
Interface	Minimum	Crucial	Curial	Minimum	Crucial
Security	Vital	Limited	Weak	Limited	High
Expertise required	High	High	Medium	Medium	High
Simplicity	Simple	Simple	Simple	Intermediate	Intermediate
User involvement	Only at beginning	At the beginning	High	At the beginning	High
Resource control	Yes	Yes	No	Yes	Yes

Conclusion

Our analysis and comparison led us to the conclusion that the models were created between 1970 and 1999. There are advantages and downsides to every model that came into being to address the issues of the prior model. As of today, waterfall and spiral are two of the most prevalent models used in the software development process, and additional models are employed as needed. According on the software's size, the models are employed by developers. To maximize software productivity and quality, developers and consumers alike focus on factors such as cheap cost, low risk, high quality, and a short development cycle.

References

[1] Barry Boehm, "Spiral Development: Experience, Principles, and Refinements", edited by Wilfred J. Hansen, 2000.

[2] Roger Pressman, Software Engineering: A Practitioner's Approach, Sixth Edition, McGraw-Hill Publication.

[3] K. K. Aggarwal, Yogesh Singh Software Engineering 3rd Edition.

[4] Karlm, "Software Lifecycle Models', KTH, 2006.

[5] Vanshika Rastogi 'Software Development Life Cycle Models-Comparison, Consequences' ISNN: 0975-9646 International Journal of Computer Science and Information Technologies, Vol. 6 (1), 2015, 168-172.

[6] Sanjana Taya "Comparative Analysis of Software Development Life" ISSN: 2229-4333 (print), ISSN: 0976-8491(online), vol.2ISSUE 4, Oct-Dec2011.

[7] Nabil Mohammed Ali Munassar And A.Govardhan, "A Comparison between Five Models of Software Engineering" Ijcsi International Journal of Computer Science Issues, Vol7, Issue 5, Sep, 2010.

[8] Comparative Analysis of Different types of Models in Software Development Life Cycle-Ms. Shikha Maheshwari, Prof. Dinesh Ch. Jain

